

Параллельные методы умножения матрицы на вектор

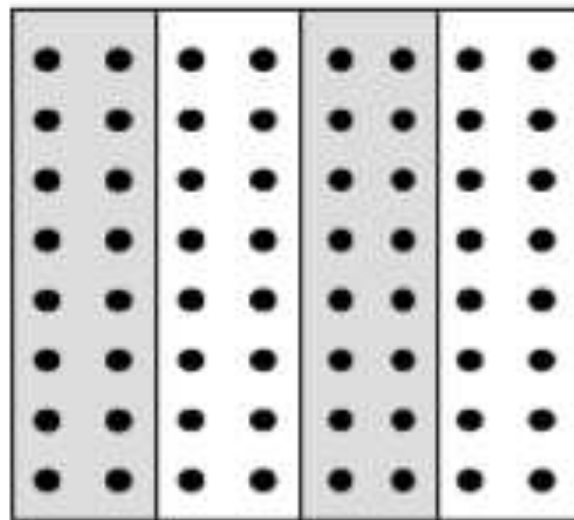
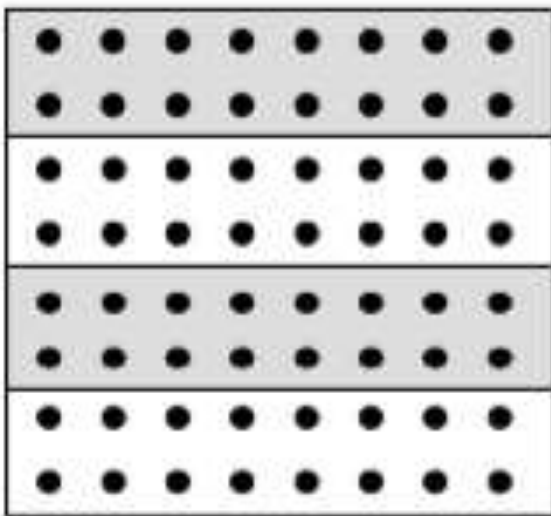
Принципы распараллеливания

- Для многих методов матричных вычислений характерным является повторение одних и тех же вычислительных действий для разных элементов матриц.
- Данный момент свидетельствует о наличии **параллелизма по данным** при выполнении матричных расчетов и, как результат, распараллеливание матричных операций сводится в большинстве случаев к разделению обрабатываемых матриц между потоками.

Принципы распараллеливания

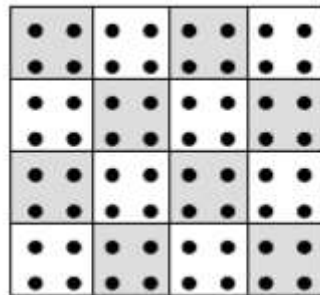
1. Ленточное разбиение матрицы

При ленточном (block-striped) разбиении каждому потоку выделяется то или иное подмножество строк (rowwise или горизонтальное разбиение) или столбцов (columnwise или вертикальное разбиение) матрицы.



2. Блочное разбиение матрицы

- При блочном (chessboard block) разделении матрица делится на прямоугольные наборы элементов – при этом, как правило, используется разделение на непрерывной основе.
- Пусть количество потоков составляет $p = s \cdot q$, количество строк матрицы является кратным s , а количество столбцов – кратным q , то есть $m = s \cdot k$ и $n = q \cdot t$. Представим исходную матрицу A в виде набора прямоугольных блоков.



Постановка задачи

- В результате умножения матрицы A размерности $m \times n$ и вектора b , состоящего из n элементов, получается вектор с размера m , каждый i -ый элемент которого есть результат скалярного умножения i -й строки матрицы A (обозначим a_i) и вектора b .

$$c_i = (a_i, b) = \sum_{j=1}^n a_{ij} b_j, \quad 0 \leq i < m.$$

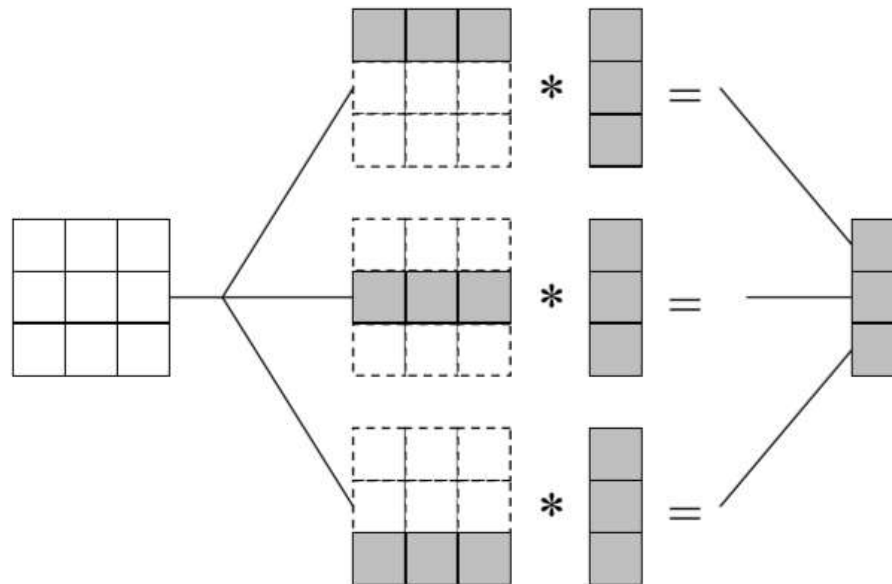
Последовательный алгоритм умножения матрицы на вектор

```
for (i = 0; i < m; i++) {  
    c[i] = 0;  
    for (j = 0; j < n; j++) {  
        c[i] += A[i][j]*b[j]  
    }  
}
```

- Для выполнения матрично-векторного умножения необходимо выполнить m операций вычисления скалярного произведения, таким образом, алгоритм имеет трудоемкость порядка $O(mn)$.

Умножение матрицы на вектор при разделении данных по строкам

- Организация вычислений при выполнении параллельного алгоритма умножения матрицы на вектор, основанного на разделении матрицы по строкам



Программная реализация (OpenMP)

```
// Function for parallel calculating matrix-vector multiplication
void ParallelResultCalculation(double* pMatrix, double* pVector,
    double* pResult, int Size) {
    int i, j; // Loop variables
    #pragma omp parallel for private (j)
    for (i=0; i<Size; i++) {
        for (j=0; j<Size; j++)
            pResult[i] += pMatrix[i*Size+j]*pVector[j];
    }
}
```

Анализ эффективности

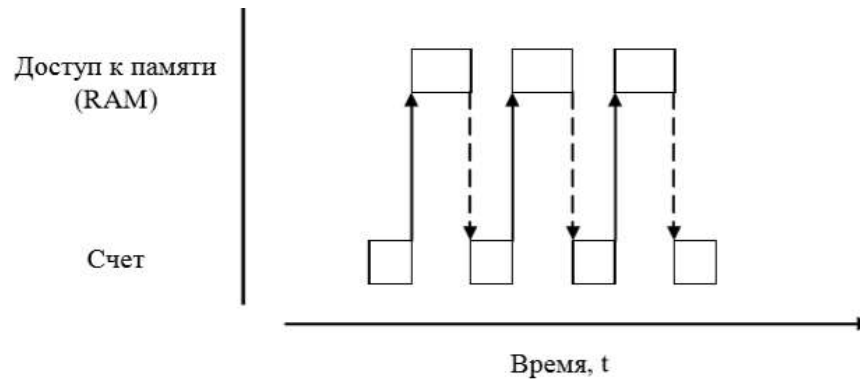


Диаграмма состояний процесса выполнения последовательного алгоритма умножения матрицы на вектор

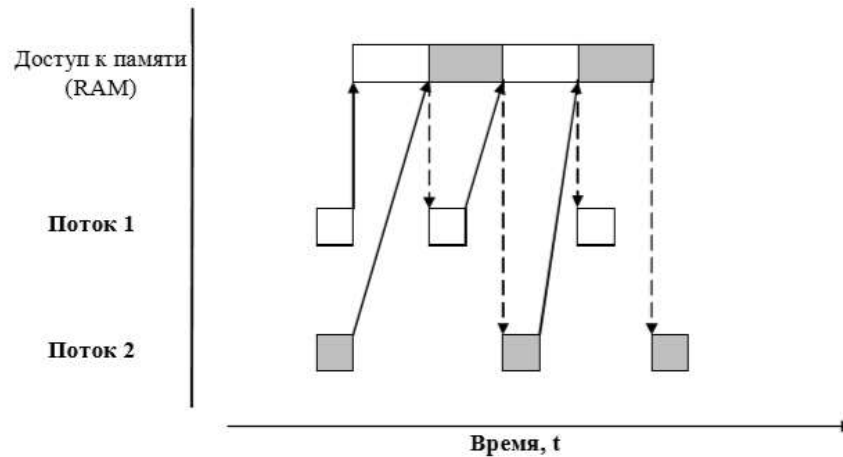


Диаграмма состояний потоков при выполнении параллельного алгоритма умножения матрицы на вектор

Анализ эффективности

Время выполнения последовательного алгоритма

- $O(n^2)$
- n – размерность матрицы

Время выполнения параллельного алгоритма

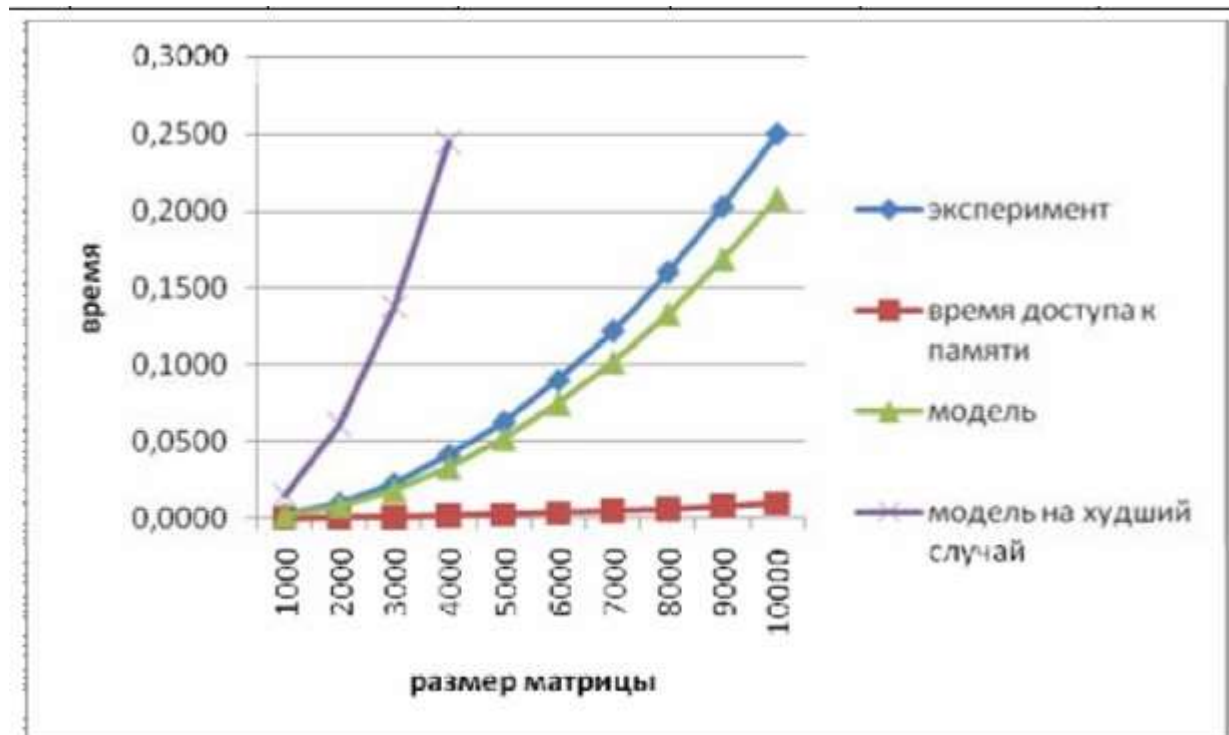
- p вычислительных
элементов, p потоков

$$T_p = \frac{n \cdot (2n - 1)}{p} \cdot t + g \cdot n^2 \cdot (a + 64 / b)$$

- a – латентность
оперативной памяти,
- b – пропускная
способность оперативной
памяти

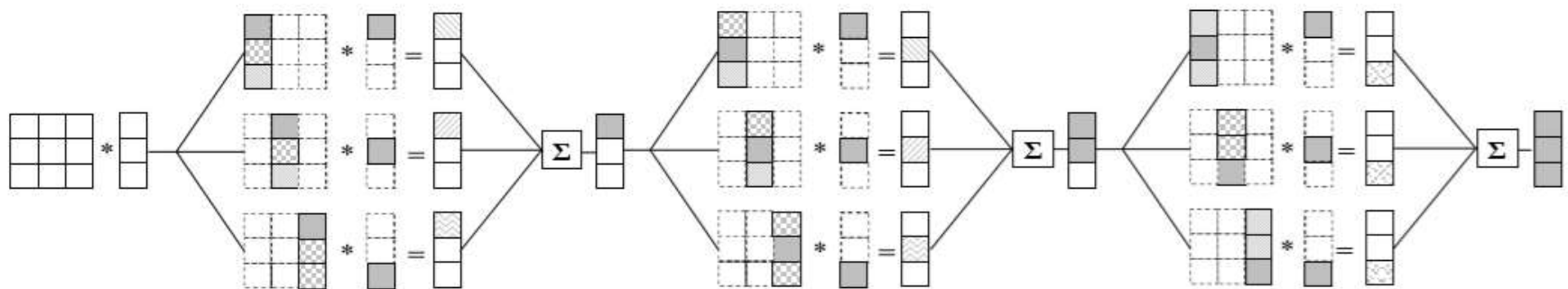
Анализ эффективности

- Intel Xeon E5320, 1.86 ГГц, 4 Гб RAM
под управлением операционной системы
Microsoft Windows HPC Server 2008.



Умножение матрицы на вектор при разделении данных по столбцам

Организация вычислений при выполнении параллельного алгоритма умножения матрицы на вектор с использованием разбиения матрицы по столбцам



Программная реализация (OpenMP)

```
// Function for parallel calculating matrix-vector multiplication
void ParallelResultCalculation(double* pMatrix, double* pVector,
    double* pResult, int Size) {
    int i, j; // Loop variables
    omp_lock_t MyLock;
    omp_init_lock(&MyLock);

    for (i=0; i<Size; i++) {
#pragma omp parallel for
        for (j=0; j<Size; j++) {
            omp_set_lock(&MyLock);
            pResult[i] += pMatrix[i*Size+j]*pVector[j];
            omp_unset_lock(&MyLock);
        }
    }
    omp_destroy_lock(&MyLock);
}
```

Анализ эффективности

- Время выполнения параллельного алгоритма умножения матрицы на вектор, основанного на вертикальном разделении матрицы, может быть вычислено для наихудшего случая (без использования кэш памяти) по формуле:

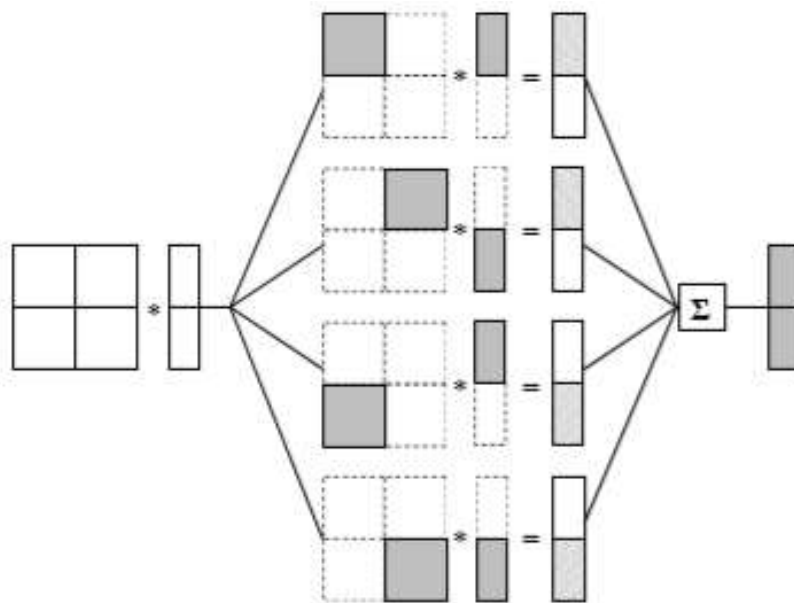
$$T_p = \left(\frac{n \cdot (2n - 1)}{p} + n \cdot \log_2 p + n \right) \cdot t + n^2 \cdot \left(a + \frac{64}{b} \right) + n \cdot d.$$

- При использовании кэш памяти с частотой кэш промахов g это соотношение принимает вид:

$$T_p = \left(\frac{n \cdot (2n - 1)}{p} + n \cdot \log_2 p + n \right) \cdot t + g \cdot n^2 \cdot \left(a + \frac{64}{b} \right) + n \cdot d.$$

Умножение матрицы на вектор при блочном разделении данных

- Общая схема параллельного алгоритма умножения матрицы на вектор при блочном разделении данных



Программная реализация (OpenMP)

- **Первый вариант.** Блок операций, следующий за объявлением критической секции, в каждый момент времени может выполняться только одним потоком. Подобная организация программы гарантирует единственность доступа потоков для изменения разделяемых данных внутри критической секции.

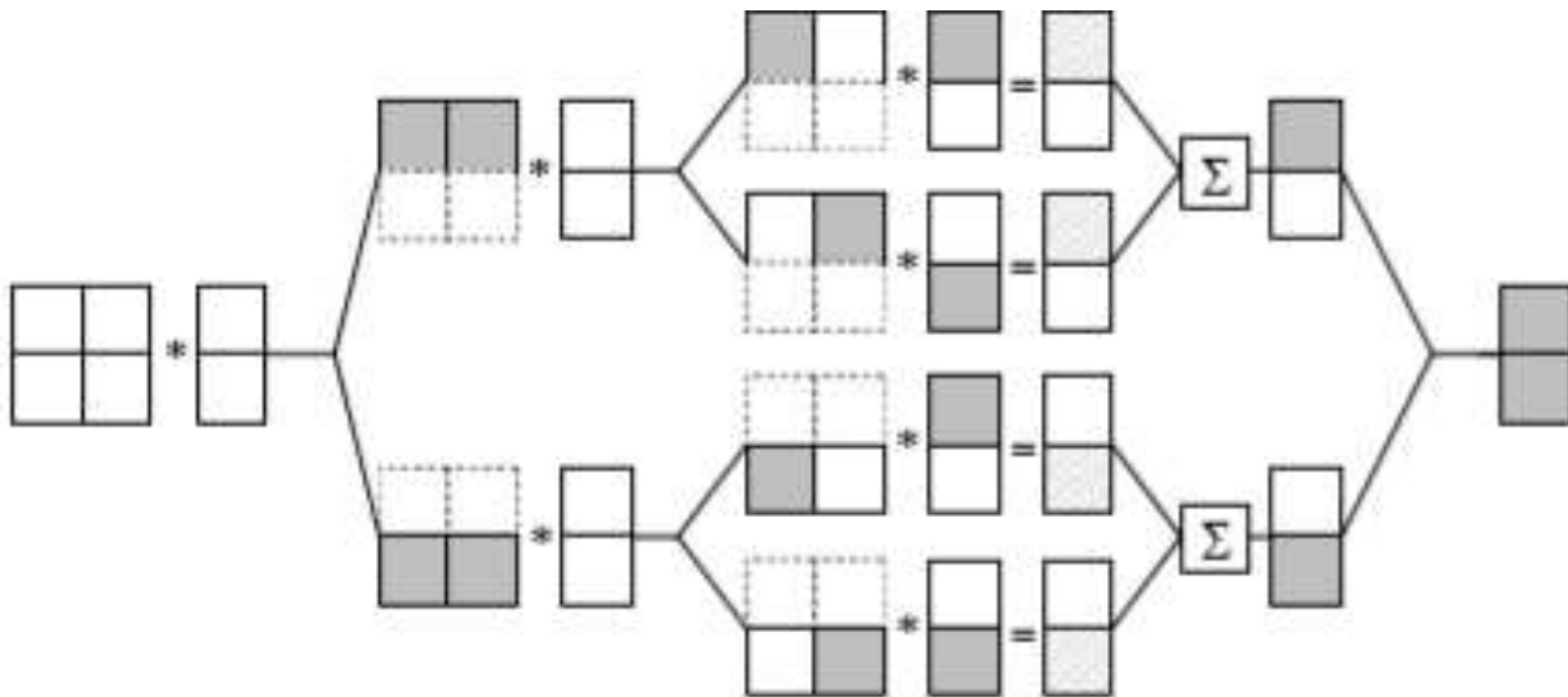
```
// Function for parallel matrix-vector multiplication
void ResultCalculation(double* pMatrix, double* pVector, double* pResult,
    int Size) {
    int ThreadID;
    int GridThreadsNum = 4;
    int GridSize = int (sqrt(double(GridThreadsNum)));
    int BlockSize = Size/GridSize; // we assume that the matrix size is
                                   // divisible by the grid size
    omp_set_num_threads(GridThreadsNum);
```

```

#pragma omp parallel private (ThreadID)
{
    ThreadID = omp_get_thread_num();
    double * pThreadResult = new double [Size];
    for (int i=0; i<Size; i++) {
        pThreadResult[i] = 0;
    }
    int i_start = (int(ThreadID/GridSize))*BlockSize;
    int j_start = (ThreadID%GridSize)*BlockSize;
    double IterResult;
    for (int i=0; i< BlockSize; i++) {
        IterResult = 0;
        for (int j=0; j < BlockSize; j++)
            IterResult +=
                pMatrix[(i+i_start)*Size+(j+j_start)] * pVector[j+i_start];
        pThreadResult[i+i_start] = IterResult;
    }
#pragma omp critical
    for (int i=0; i<Size; i++) {
        pResult[i] += pThreadResult[i];
    }
    delete [] pThreadResult;
} // pragma omp parallel
}

```

- **Второй вариант.** Стандарт OpenMP предусматривает возможность организации вложенных параллельных секций. Это значит, что внутри одной параллельной секции можно объявить вторую параллельную секцию, которая разделит каждый из потоков внешней параллельной секции на несколько вложенных потоков.



```
// Function for matrix-vector multiplication
void ResultCalculation(double* pMatrix, double* pVector, double* pResult,
    int Size) {
    int NestedThreadsNum = 2;
    omp_set_num_threads(NestedThreadsNum);
    omp_set_nested(true);
    #pragma omp parallel for
    for (int i=0; i<Size; i++) {
        double ThreadResult = 0;
        #pragma omp parallel for reduction(+:ThreadResult)
        for (int j=0; j<Size; j++)
            ThreadResult += pMatrix[i*Size+j]*pVector[j];
        pResult[i] = ThreadResult;
    }
}
```

- В приведенной программе для задания количество потоков, создаваемых на каждом уровне вложенности параллельных областей, используется переменная ***NestedThreadsNum*** (в данном варианте программы ее значение устанавливается равным 2).

Анализ эффективности

- Первый вариант реализации.

$$T_p = \left(\frac{n \cdot (2n - 1)}{p} + n \cdot p \right) \cdot t + g \cdot n^2 \cdot \left(a + \frac{64}{b} \right).$$

- Второй вариант реализации.

$$T_p = \left(\frac{n \cdot (2n - 1)}{p} + \frac{n}{q} \cdot \log_2 q + \frac{n}{q} \right) \cdot t + g \cdot n^2 \cdot \left(a + \frac{64}{b} \right) + \frac{n}{q} \cdot d$$

Показатели ускорения рассмотренных параллельных алгоритмов умножения по результатам вычислительных экспериментов

